PROJECT AND TEAM INFORMATION

Project Title

DYNAMIC SMART CITY ENGINE

A Dynamic Incident Prioritization and Resource Coordination System for Urban Services

Student / Team Information

Team ID	DSCPP-III-2026-T374	**Semester**	3rd
Course	Data Structure and Programming in C++	**Mentor**	Mr. Kireet Joshi

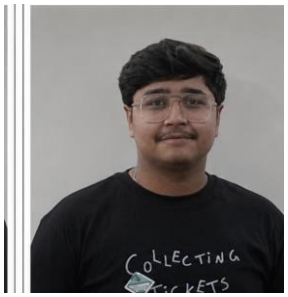
Name	Roll No.	Section	Role	College Email
Pavani Jain	260222072	DS1	Team Leader	260222072@geu.ac.in
Karan Babbar	2029058	SEC-B	Team Member	2510360011@geu.ac.in
Manan Kumar	2028450	DS2	Team Member	2510380086@geu.ac.in

Team Members



Pavani Jain

260222072@geu.ac.in



Karan Babbar

2510360011@geu.ac.in



Manan Kumar

2510380086@geu.ac.in

PROPOSAL DESCRIPTION

Motivation

In a city, different problems can be reported at the same time. For example, a traffic problem and an emergency may arrive together, but both should not be treated in the same way. Our idea is to give the incidents a priority and then decide which available resource can be used. After that, the system can find a route for the selected resource. We chose this problem because it lets us use the C++ and data structure concepts from the course in one project.

State of the Art / Current Solution

City-service information can be collected through different systems, but the decision of which incident should be handled first, which resource should be used and which route should be followed can still involve separate steps. Our project is a small academic prototype. It does not try to replace a real city system. We are using sample incidents, sample resources and a small road graph to demonstrate the decision process.

Project Goals and Milestones

- The main aim of our project is to develop a C++ system for managing city incidents.
- • Record and organize incoming incidents.
- • Give priority to incidents using severity, impact and waiting time.
- • Apply linked list, queue, priority queue, hashing, BST, sorting and stack in the incident module.
- • Maintain resource details and check their type, status and location.
- • Choose an available and suitable resource for an incident.
- • Represent city locations and roads using a weighted graph.

• Use BFS, DFS and Dijkstra for route-related operations.

• Test the three modules separately and combine them after testing.

• Complete edge-case checking, documentation and the final demonstration.

Module 1 — Incident & Dynamic Priority Engine

This part handles the incidents. We created a base class called CityEvent and derived classes for Traffic, Waste, Infrastructure and Emergency events. The priority is calculated from severity, impact and waiting time. The priority queue helps us pick the more urgent incident first. We use a linked-list container to keep incident records and a normal queue to keep their arrival order. An unordered map is used for quick ID lookup. A BST is used for ID indexing, insertion sort is used when we need an ordered display, and a stack stores recent status changes.

Module 2 — Resource & Infrastructure Management

This part keeps the information about available resources. A resource has an ID, type, location, status and capacity. For an incident, we first check the resource type and availability. If more than one resource can be used, we compare their graph distances and choose the nearer reachable one. The resource information can also be saved in CSV format.

Module 3 — City Graph & Routing Engine

This part represents the city as a graph. Locations are vertices and roads are weighted edges. We store the graph using an adjacency list. BFS and DFS are used for traversal. Dijkstra's algorithm is used when we need the shortest route between two locations. The parent information is used to show the final path.

System Architecture (High Level Diagram)

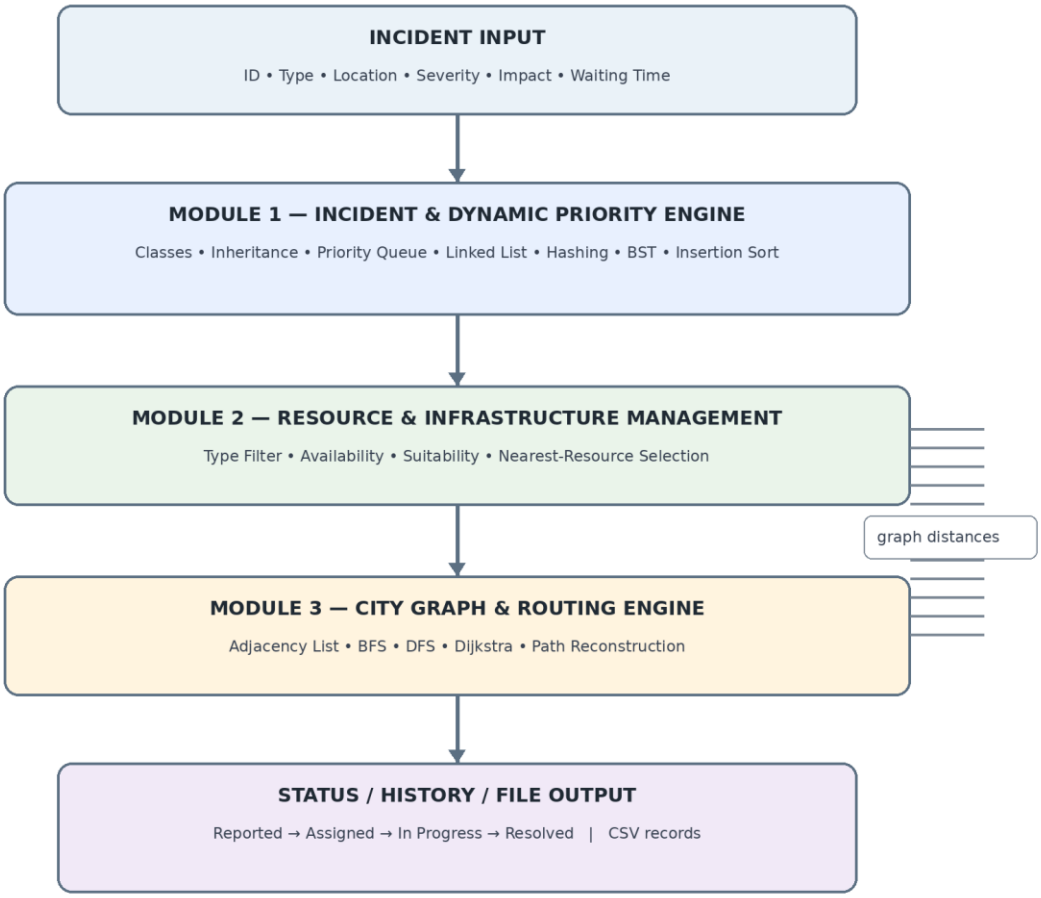


Figure 1. System architecture prepared using draw.io.

PROJECT OUTCOME / DELIVERABLES

At the present stage, the project shows how an incident can move through the three main parts of the system. The incident is stored and given a priority. A suitable resource can then be selected and a route can be found on the city graph. The project uses sample data, so the locations, resources and road distances are only for demonstration.

Sample Result

In the sample case, emergency incident 102 is at ISBT and gets a priority value of 7.00. The available ambulance at Ballupur is selected. The route shown by the graph is Ballupur → Clock Tower → ISBT, with a simulated distance of 11 units.

Expected Deliverables

- C++ source code for the three modules and the final integrated program.
- Unit tests for the incident, resource and graph modules.
- Sample CSV data and sample output.
- README containing project, setup and module information.
- GitHub repository showing the individual work of the team members.
- Phase-II report and presentation.

Assumptions

The project uses sample data instead of live city data. Road distances are fixed and non-negative. Resource availability is stored as a status. The priority weights are selected by our team for this prototype. The program does not use live GPS, sensors, government systems or map services.

Technical Notes

Concept	How we use it
Priority Queue	Chooses the more urgent pending incident first.
Hashing	Helps find an incident using its ID.
BST	Keeps an ordered index for incident IDs.
Linked List	Stores the incident records.
Queue	Keeps the arrival order of incidents.
Graph + Dijkstra	Finds a shortest route using the road distances.

REFERENCES

1. Course material: Data Structure and Programming in C++ (Graphic Era, 2026–27).

2. cppreference — C++ Standard Library documentation.

3. Cormen, Leiserson, Rivest and Stein, Introduction to Algorithms.

4. Project GitHub repository: pavanijain/dynamic_smart_city_engine.

PROJECT MODULE RESPONSIBILITY

Member	Module	Main work
Pavani Jain	Module 1 — Incident & Dynamic Priority Engine	OOP, inheritance, polymorphism, linked list, queue, priority queue, hashing, BST, sorting, stack
Manan Kumar	Module 2 — Resource & Infrastructure Management	Resource records, filtering, availability, nearest-resource selection, CSV
Karan Babbar	Module 3 — City Graph & Routing Engine	Adjacency list, BFS, DFS, Dijkstra, path reconstruction

Note: This editable version contains the same revised report content. You can directly change any sentence yourself before submitting.